

TRUELENS

Verifying Authenticity in the Age of AI

FULL-STACK WEB APPLICATION COMPLETE IMPLEMENTATION PLAN

Team: Aryam Agarwal • Priyanshu Agarwal • Hitarth Singh Rajput • Akshat Singh • Piyush Kumar
Version 1.0 • May 2026

Executive Summary

TrueLens is a unified AI-powered web platform designed to detect and flag AI-generated content, fake reviews, manipulated images, bot-amplified pages, and fraudulent documents. Built with a modern full-stack architecture and a rich dark-purple aesthetic, TrueLens delivers authenticity scores, cryptographic document signing, and a public API — making the internet more trustworthy one scan at a time.

Document Overview

This implementation plan is organized into 5 iterative phases. Each iteration builds on the last, ensuring a working prototype is available after Iteration 1, with subsequent iterations adding depth, polish, and production-readiness.

Phase	Description
Iteration 1	Core Prototype — working MVP with frontend + basic ML backend
Iteration 2	Full ML Pipeline — integrate all detection models
Iteration 3	Document Tamper Protection — cryptographic signing workflow
Iteration 4	TrueLens API — public REST API + platform integrations
Iteration 5	Polish & Production — performance, security, scale, launch

Technology Stack

Every technology choice has been made to maximize developer velocity, minimize cost, and ensure the result is a credible, production-quality application.

Frontend

Layer	Choice & Rationale
Framework	Next.js 14 (App Router) — file-based routing, SSR/SSG, React Server Components
UI Library	shadcn/ui + Tailwind CSS — headless, accessible, fully customizable components
Animations	Framer Motion — smooth transitions matching the premium brand aesthetic
State	Zustand (client) + TanStack Query (server state + caching)
Charts	Recharts — trust score gauges, history charts, signal breakdowns
Icons	Lucide React — consistent icon set
Forms	React Hook Form + Zod — type-safe form validation
File Upload	react-dropzone — drag-and-drop file uploads for document analysis
Auth UI	NextAuth.js — Google/GitHub OAuth + email magic link

Backend

Layer	Choice & Rationale
Runtime	Node.js 20 LTS + Express.js — familiar, fast, huge ecosystem
Language	TypeScript throughout — type safety across the full stack
API Style	REST + JSON (v1 API) with OpenAPI 3.0 spec auto-generated
Auth	JWT (access + refresh tokens) + bcrypt for password hashing
File Storage	AWS S3 (or Cloudflare R2 for cost) — stores uploaded content for analysis
Queue	BullMQ + Redis — async deep-check job processing
Caching	Redis — cache analysis results, rate-limit counters, session data
Email	Resend API — transactional emails for verification, alerts
Webhooks	For API customers: retry-safe webhook delivery via BullMQ

ML / AI Pipeline

Component	Tech & Model
Text Detection	Python FastAPI microservice, DistilBERT fine-tuned on HC3 dataset (HuggingFace)
Image Forensics	Python service, CNNSpec/GAN fingerprint detector + EXIF extraction via exiftool
Fake Review	Fine-tuned RoBERTa on Amazon/Yelp fake review datasets + stylometry features
Fact Checking	Google Fact Check API + NLP claim extraction
Provenance	TinEye API + Google Image Search reverse-lookup + Wayback Machine API
Domain Trust	WHOIS lookup + Alexa/SimilarWeb traffic API + custom reputation DB
Ensembling	Weighted scoring function: 0-100 trust score aggregated from sub-scores
Serving	FastAPI + uvicorn, containerized with Docker
Training	Hugging Face Transformers + PyTorch; pre-trained weights for MVP

Database

Purpose	Technology
Primary DB	PostgreSQL 16 — users, scans, reports, API keys, document records
ORM	Prisma — type-safe queries, migrations, schema-as-code
Search	PostgreSQL full-text search (upgrade to Typesense in Iteration 5)
Cache/Queue	Redis 7 — BullMQ jobs, caching, sessions, rate limiting
File Storage	AWS S3 / Cloudflare R2 — uploaded documents, images, evidence packs
Vector Store	pgvector extension — semantic similarity for content matching (Iteration 4)

Infrastructure & DevOps

Service	Technology
---------	------------

Containerization	Docker + Docker Compose (dev) → Kubernetes or ECS (prod)
CI/CD	GitHub Actions — lint, test, build, deploy on every push
Hosting	Vercel (frontend) + Railway or Render (backend + ML services) — free tier friendly
Monitoring	Sentry (errors) + Posthog (analytics) + Prometheus + Grafana (metrics)
Secrets	Doppler or AWS Secrets Manager
DNS/SSL	Cloudflare — free SSL, DDoS protection, CDN
Logging	Winston (Node.js) + Loguru (Python) → aggregated in Grafana Loki

Design System & Aesthetic

TrueLens uses a rich, dark-mode-first design language: deep blacks and purples with glowing accent highlights, evoking precision, trust, and cutting-edge technology.

Color Palette

Token	Hex Value
--bg-primary	#09090B (near-black background)
--bg-card	#18181B (dark card surfaces)
--brand-deep	#6B21A8 (deep purple — headers, borders)
--brand-mid	#7C3AED (primary purple — CTAs, highlights)
--brand-light	#A78BFA (light purple — subheadings, icons)
--brand-glow	#8B5CF6 (accent purple — glows, hover states)
--text-primary	#FAFAFA (primary text)
--text-muted	#71717A (secondary/muted text)
--success	#10B981 (authentic/verified — green)
--danger	#EF4444 (flagged/AI-generated — red)
--warning	#F59E0B (uncertain — amber)

Typography

Usage	Font & Style
Display / Hero	Inter or Geist — 72-96px, bold, white
Section Headings	Inter — 36-48px, bold, white or light-purple
Body Text	Inter — 16px, 400 weight, muted white (#FAFAFA)
Code / Hashes	JetBrains Mono — 14px, light purple
Labels / Badges	Inter — 12px uppercase, letter-spacing 0.1em

Key UI Components

- Trust Score Gauge — circular SVG gauge, color-coded red/amber/green, animated on load

- Signal Cards — collapsible evidence cards for each detection signal (text, image, metadata)
- Scan Timeline — history of scans per user with trend sparklines
- Dark Glassmorphism Cards — backdrop-blur, translucent purple-tinted surfaces
- Gradient Hero — animated purple-to-black mesh gradient on landing page
- Animated Scan Loader — pulsing TrueLens logo while analysis runs
- Document Signature Badge — locked/unlocked padlock with QR code for verified docs
- Browser Extension Popup — compact floating card with trust score and key signals

Database Schema

The following core tables power the TrueLens platform. Defined in Prisma schema syntax and migrated to PostgreSQL.

Core Tables

Table	Key Columns & Purpose
users	id, email, name, avatar, plan (free/pro/enterprise), createdAt, updatedAt
scans	id, userId, url, contentType, rawText, trustScore, verdict, signalsJson, status, createdAt
signals	id, scanId, type (text/image/metadata/review/domain), score, evidenceJson
documents	id, userId, filename, s3Key, hash, status (pending/verified/flagged), signatureJson, createdAt
api_keys	id, userId, keyHash, name, scopes, rateLimit, lastUsedAt, expiresAt
reputation_db	id, domain, trustScore, category, sources, lastCheckedAt
review_scans	id, scanId, platform, productUrl, totalReviews, fakeCount, confidenceScore
jobs	id, scanId, type, status, payload, result, attempts, createdAt, processedAt
audit_logs	id, userId, action, resourceType, resourceId, meta, ip, createdAt

All tables include soft-delete (deletedAt), indexes on foreign keys and frequently queried columns. Sensitive fields (API keys) are stored as bcrypt hashes, never plaintext.

Core Prototype

Working MVP — Frontend + Basic Backend + Initial ML

Goal: A fully working prototype where a user can paste a URL or text, receive a trust score with basic signal breakdown, and view a styled results page. This is the demo-ready milestone.

1.1 Project Scaffolding

1. Initialize monorepo with pnpm workspaces: /apps/web (Next.js), /apps/api (Express), /services/ml (FastAPI)
2. Set up ESLint, Prettier, Husky pre-commit hooks, TypeScript strict mode across all packages
3. Create Docker Compose file with services: postgres, redis, api, ml, web
4. Set up GitHub repo with branch protection, Actions pipeline (lint → test → build)
5. Configure environment variable management with .env.example files for all services
6. Initialize Prisma with PostgreSQL — create initial migration with users, scans, signals tables

1.2 Frontend — Landing Page & Core UI

7. Build Next.js app with App Router, configure Tailwind CSS with TrueLens design tokens (custom CSS variables)
8. Install and configure shadcn/ui components: Button, Card, Input, Badge, Tabs, Tooltip, Dialog, Progress
9. Install Framer Motion — build animated hero section with mesh gradient background
10. Landing page sections: Hero (animated), Problem statement, How It Works (3-step), Core Capabilities grid, CTA
11. Build NavBar: TrueLens logo (SVG), links, Sign In button; mobile hamburger menu
12. Build main Scan page: URL input field + Text paste textarea with tab switcher, Submit button with loading state
13. Build Results page: Trust score gauge (SVG, animated), Verdict badge (Authentic/Suspicious/AI-Generated), Signal breakdown accordion
14. Set up NextAuth.js with Google OAuth provider; add Sign In/Sign Up pages; protect /dashboard route
15. Build User Dashboard: scan history table with pagination, summary stats cards

1.3 Backend API — Core Endpoints

16. Express app setup: cors, helmet, compression, morgan logging, rate limiting (express-rate-limit + Redis)
17. Auth middleware: JWT verification, refresh token rotation, NextAuth session bridge
18. POST /api/v1/scans — accepts {url?, text?, file?}, enqueues deep-check job, returns scanId
19. GET /api/v1/scans/:id — returns scan result (polls until complete); SSE endpoint for live progress
20. GET /api/v1/scans — paginated scan history for authenticated user
21. POST /api/v1/auth/register, POST /api/v1/auth/login, POST /api/v1/auth/refresh, POST /api/v1/auth/logout
22. GET /api/v1/health — health check endpoint
23. Error handling middleware: consistent JSON error responses, Sentry integration

1.4 ML Service — Basic Text Detection

24. Set up FastAPI app with uvicorn, Pydantic models for request/response validation
25. Install transformers, torch, scikit-learn, numpy, pandas via pip
26. Load pre-trained DistilBERT fine-tuned for AI text detection (use roberta-base-openai-detector from HuggingFace as baseline)
27. POST /analyze/text — accepts text string, returns {score: float, label: str, confidence: float, highlights: []}
28. POST /analyze/url — fetches URL content (requests + BeautifulSoup), extracts text, runs text analysis
29. Add stylometry features: avg sentence length, punctuation density, vocabulary richness, burstiness — combine with transformer score
30. Return paragraph-level breakdown so frontend can highlight suspicious sections

1.5 Integration & BullMQ Job Queue

31. Set up BullMQ with Redis: define ScanQueue, ScanWorker
32. ScanWorker orchestrates: call ML service /analyze/text, collect basic domain signals (WHOIS via whois npm package)
33. Weighted scoring: $\text{text_score} * 0.6 + \text{domain_score} * 0.4 = \text{trust_score}$ (0-100)
34. Write results to PostgreSQL scan + signals tables
35. Emit job progress events via Redis pub/sub → SSE endpoint

1.6 Iteration 1 Deliverables

- Fully functional web app accessible at localhost:3000
- User can scan any URL or paste text and receive a trust score within ~5 seconds
- Results page shows trust score gauge, verdict, text analysis signal card
- User auth (Google OAuth) working, scan history saved and visible in dashboard
- All services containerized and running via docker-compose up
- README with setup instructions and architecture diagram

PROTOTYPE CHECKPOINT: After Iteration 1, a demo-ready prototype exists. Show this to stakeholders, collect feedback, and prioritize Iteration 2 features accordingly.

Full ML Pipeline

Image Forensics + Fake Reviews + Bot Detection + Provenance

Goal: Integrate all planned detection signals into the ML pipeline. The trust score now reflects multi-modal analysis. Users get a rich evidence pack with detailed reasoning for each signal.

2.1 Image Forensics Service

- 36. Install Pillow, opencv-python, exifread, imagehash, onnxruntime in ML service
- 37. EXIF extraction: parse all metadata fields; flag missing/stripped EXIF (common in AI images), inconsistent timestamps
- 38. Error Level Analysis (ELA): re-compress image at known quality, compare pixel differences — reveals manipulation regions
- 39. GAN fingerprint detection: load pre-trained CNNDetection model (Gragnaniello et al.) via ONNX for fast inference
- 40. Image hash (pHash) computation for reverse-lookup in provenance step
- 41. POST /analyze/image — accepts base64 image or URL, returns {ganScore, elaScore, exifSignals, overallScore, evidenceMap}
- 42. Frontend: Image signal card shows ELA heatmap overlay, EXIF table, GAN confidence bar

2.2 Fake Review Detection

- 43. Fine-tune RoBERTa on combined dataset: Amazon Mechanical Turk fake reviews + Yelp filtered reviews dataset (available on HuggingFace: raj-tomar001/fake-reviews-dataset)
- 44. Features: transformer embeddings + stylometry (all-caps ratio, exclamation count, review length, verified purchase flag, reviewer history velocity)
- 45. POST /analyze/reviews — accepts {url: string} (product/business page), scrapes visible reviews, returns per-review score + aggregate stats
- 46. Implement ethical scraping: respect robots.txt, rate limit, use rotating user agents, cache results 24h
- 47. Frontend: Review signal card shows pie chart (authentic vs fake %), flagged review excerpts with confidence scores

2.3 Bot Amplification & Domain Trust

- 48. Domain signal pipeline in Node.js worker: WHOIS age (new domains flagged), SSL certificate check, Alexa/SimilarWeb rank
- 49. Social signal analysis: detect unnatural share velocity patterns if social API data available
- 50. Reputation database: seed with known bad domains (PhishTank, URLhaus, OpenPhish feeds); good domains (Majestic Million)
- 51. POST /analyze/domain — returns {age, rank, reputationScore, blacklistHits, sslValid}
- 52. Frontend: Domain card shows trust badge, domain age, traffic rank, blacklist status

2.4 Provenance & Reverse Search

- 53. Integrate TinEye API (free tier 100 searches/month, upgrade for production) for reverse image search
- 54. Google Custom Search API for text plagiarism: search distinctive phrases, find earliest indexed occurrence
- 55. Wayback Machine API: check if URL/content existed before claimed publication date
- 56. Implement result caching (Redis, 1h TTL) to avoid redundant API calls
- 57. POST /analyze/provenance — returns {earliestDate, sources: [], imageMatches: [], plagiarismHits: []}

2.5 Ensemble Scoring Engine

- 58. Define configurable weight matrix (stored in DB, admin-adjustable):
 - Text AI detection: 30%
 - Image forensics: 25%
 - Fake review detection: 20%
 - Domain trust: 15%
 - Provenance: 10%
- 59. Signals only included in score if the content type has them (e.g., image score skipped for text-only pages)
- 60. Confidence-weighted: low-confidence signals contribute less to final score
- 61. Every verdict includes which signals fired and why — human-readable explanation strings

2.6 Enhanced Frontend

- 62. Results page: tabbed signal breakdown (Text, Image, Reviews, Domain, Provenance) — each with charts and evidence
- 63. Trust score history chart (line chart) on user dashboard — see how a URL's score changes over time

- 64. Comparison view: scan two URLs side by side
- 65. Export report as PDF (use Puppeteer serverside to render results page → PDF)
- 66. Shareable scan results: public URL with scan ID (/results/[scanId]) — no auth required to view

Iteration 2 produces a genuinely useful product. The multi-signal approach dramatically reduces false positives. Run internal accuracy testing on a labeled dataset before deploying to production.

Document Tamper Protection

Cryptographic Signing + Document Verification Workflow

Goal: Build the document verification and tamper-evident signing pipeline. Support Aadhaar cards, invoices, medical prescriptions, and contracts. Issue verifiable cryptographic signatures for clean documents.

3.1 Document Upload & Storage

67. Secure file upload endpoint: POST /api/v1/documents/upload — multipart form, max 25MB
68. Accepted formats: PDF, PNG, JPG, JPEG, DOCX — validate MIME type server-side (not just extension)
69. Virus scan on upload: integrate ClamAV or VirusTotal API before storing
70. Store in S3 with server-side AES-256 encryption; generate pre-signed URLs for ML service access
71. Compute SHA-256 hash of file immediately on receipt — store in DB as document fingerprint
72. Implement upload rate limiting: 10 documents/hour for free plan, 100/hour for Pro

3.2 Document ML Analysis

73. PDF parsing: use pdfplumber (Python) to extract text, images, and form fields from PDFs
74. OCR pipeline: use Tesseract (pytesseract) for scanned documents; EasyOCR as fallback for handwritten content
75. Document-specific classifiers:
 - Aadhaar Card: validate QR code structure, check font consistency, detect copy-paste artifacts on name/DOB fields
 - Invoice: verify arithmetic consistency (line items sum to total), detect cloned fields, validate GST/VAT number format
 - Medical Prescription: detect missing required fields, validate doctor registration number format, check date logic
 - Contract: detect inconsistent fonts/signatures, validate party name consistency across clauses
76. Image forensics on embedded images within documents (signature blocks, stamps, photos)
77. POST /analyze/document — returns {manipulationScore, findings: [], documentType, extractedFields}

3.3 Cryptographic Signing Infrastructure

- 78. Generate RSA-4096 or Ed25519 key pair for TrueLens signing authority; store private key in AWS KMS or HashiCorp Vault (never in code or env files)
- 79. Signing payload: {documentHash, scanId, verdict, timestamp, documentType, signerPublicKey}
- 80. Sign payload with private key → produce base64-encoded signature
- 81. Generate verifiable evidence pack: JSON file containing {documentHash, signature, trustScore, signals, publicKey, timestamp}
- 82. Store evidence pack in S3; attach download URL to document record
- 83. Issue signed document: embed invisible metadata watermark in PDF (using pypdf) containing signature and TrueLens verification URL

3.4 Verification Portal

- 84. Public verification page: /verify — user uploads or pastes document hash/scan ID
- 85. System checks: hash matches original scan, signature valid against public key, document not subsequently flagged
- 86. Show verification result: green Verified badge with timestamp, or red Invalid/Flagged badge with reason
- 87. QR code on signed documents links directly to /verify/[scanId] — instant verification for third parties
- 88. Bulk verification API endpoint for enterprise customers: POST /api/v1/verify/batch

3.5 Frontend — Document Workflow

- 89. New /documents section in dashboard: upload area with drag-and-drop, document type selector, analysis progress
- 90. Document detail page: extracted fields display, manipulation heatmap overlay, findings list, signature download
- 91. Verification badge component: embeddable HTML snippet for verified documents (like Google Scholar badges)
- 92. Print-ready verification certificate: PDF with TrueLens branding, QR code, document hash, timestamp

Security note: The private signing key must never touch application code. Use AWS KMS SigningJob API or equivalent HSM. Log all signing operations to immutable audit trail.

Goal: Open the platform to external developers and platforms. Build and publish the browser extension. Enable X/WhatsApp-style content flagging integrations.

4.1 Public REST API

- 93. Design OpenAPI 3.0 spec; auto-generate from Express routes using swagger-jsdoc
- 94. API key management: generate, rotate, revoke via /api/v1/developer dashboard; keys stored as bcrypt hashes
- 95. Rate limiting tiers: Free 100 req/day, Pro 10,000 req/day, Enterprise unlimited with SLA
- 96. Core API endpoints:
 - POST /v1/scan — submit URL, text, or file for analysis; returns {scanId, estimatedTime}
 - GET /v1/scan/{scanId} — poll for result; returns full verdict JSON
 - POST /v1/scan/sync — synchronous scan (≤ 3 s timeout, text only) for real-time integrations
 - POST /v1/verify — verify a document by hash or scanId
 - GET /v1/domain/{domain} — domain trust score lookup
 - GET /v1/usage — API usage stats for current key
- 97. Standardized JSON verdict format: {scanId, url, trustScore, verdict, confidence, signals: [], timestamp, scanDuration}
- 98. Webhook delivery: POST to customer-configured URL on scan completion; HMAC-SHA256 signature on payload
- 99. SDK packages: publish @truelens/node and @truelens/python to npm/PyPI (thin wrappers around REST API)

4.2 Browser Extension (Chrome + Firefox)

- 100. Create extension using Plasmo framework (React-based extension builder, works for Chrome + Firefox from one codebase)
- 101. Manifest V3 compliant; request only necessary permissions: activeTab, storage, identity
- 102. Extension popup: compact TrueLens card with trust score gauge, top 3 signals, link to full web report
- 103. Quick-check on page load: call TrueLens API sync endpoint with current URL, show score in toolbar badge (color-coded)

- 104. Context menu integration: right-click → "Analyze with TrueLens" on selected text or images
- 105. User settings in extension: auto-scan on/off, minimum score threshold for alert, API key input for Pro users
- 106. Content script: inject subtle TrueLens indicator banner on pages with score < 40 (highly suspicious)
- 107. Build and publish to Chrome Web Store and Firefox Add-ons Marketplace

4.3 API Gateway & Multi-Modal Pipeline

- 108. API Gateway layer (from architecture diagram): Client Request Ingestion → Authentication → Input Schema Validation
- 109. Pre-Processing Engine: Text Normalization, Language Identification (langdetect), Tokenization & Embedding (sentence-transformers)
- 110. Multi-Modal Analysis Core: AI Detection, Fact-Checking, Text Normalization, Review Detection — run in parallel via asyncio
- 111. Aggregator & Decision Logic: Weighted Scoring Engine → Final Verdict Categorization → JSON response
- 112. Support multilingual content: Hindi, Tamil, Bengali in addition to English — use multilingual BERT models

4.4 Developer Portal

- 113. Build /developers section: API documentation (auto-generated from OpenAPI spec using Redoc), interactive API explorer
- 114. API key dashboard: create/revoke keys, view usage charts, set webhook URLs
- 115. Code examples in Node.js, Python, curl for every endpoint
- 116. Sandbox environment: free test API key with synthetic responses — no real ML calls, for integration testing
- 117. Changelog page: versioned API updates, deprecation notices

Platform integration note: X (Twitter) and WhatsApp do not have open APIs for content injection. Position TrueLens API for browser extensions, fact-checker tools, journalism platforms, and enterprise document workflows — these are the realistic B2B channels.

Goal: Harden the platform for production traffic. Optimize performance, implement comprehensive security measures, add monitoring and alerting, and prepare for public launch.

5.1 Performance Optimization

- 118. Frontend: Next.js Image optimization, code splitting, lazy loading for heavy components (charts, maps)
- 119. Implement Suspense boundaries with skeleton loaders for all async data fetches
- 120. Redis caching strategy: cache domain trust scores (1h), scan results for identical URLs (24h), reputation DB lookups (6h)
- 121. ML service optimization: model quantization (INT8) for 2-4x inference speedup; ONNX export for production models
- 122. Database: add composite indexes on (userId, createdAt), (url, createdAt); analyze slow queries with EXPLAIN ANALYZE
- 123. Implement HTTP/2 + compression (brotli) on API server
- 124. CDN for static assets and pre-rendered pages via Vercel/Cloudflare

5.2 Security Hardening

- 125. Security headers: CSP, HSTS, X-Frame-Options, X-Content-Type-Options, Permissions-Policy via next/headers and helmet
- 126. Input sanitization: DOMPurify for any user HTML, parameterized queries everywhere (Prisma handles this), Zod validation on all API inputs
- 127. File upload security: re-validate MIME type after upload, scan with ClamAV, restrict S3 bucket to server-only access (no public S3 URLs)
- 128. SQL injection: Prisma ORM prevents this by design — document and enforce no raw SQL
- 129. SSRF prevention: validate and allowlist/blocklist URLs before fetching (block localhost, 169.254.x.x, RFC 1918 ranges)
- 130. Authentication: implement MFA (TOTP) for Pro/Enterprise accounts; session invalidation on password change
- 131. API key security: store only bcrypt hash, show key only once at creation, support key scoping (read-only keys)
- 132. Dependency audit: npm audit, pip-audit in CI pipeline; Dependabot for automated PRs

133. Penetration test checklist: OWASP Top 10, test all endpoints with authenticated and unauthenticated requests

5.3 Monitoring & Observability

134. Sentry: error tracking for Next.js (frontend) and Express (backend); set up alerts for error rate spikes
135. Prometheus metrics: scan throughput, ML inference latency, queue depth, API error rates — expose /metrics endpoint
136. Grafana dashboards: operational dashboard (uptime, latency, error rate), ML dashboard (model accuracy drift), business dashboard (scans/day, new users)
137. Alerting: PagerDuty or Slack alerts for: API error rate > 1%, ML service down, queue depth > 1000, disk > 80%
138. Distributed tracing: OpenTelemetry across Next.js → Express → ML service — trace full scan lifecycle
139. Uptime monitoring: Better Uptime or Uptime Robot for public status page (status.truelens.app)

5.4 Scalability & Infrastructure

140. Horizontally scale Express API with multiple instances behind a load balancer (Nginx or AWS ALB)
141. ML service: scale GPU/CPU replicas based on BullMQ queue depth (Kubernetes HPA or Railway autoscale)
142. Database connection pooling: PgBouncer between Express and PostgreSQL
143. BullMQ: separate queues for quick-check (high priority, short TTL) vs deep-check (lower priority, longer TTL)
144. Multi-region consideration: deploy to 2 regions (e.g., us-east-1 + eu-west-1) for latency and GDPR compliance
145. Implement database read replicas for scan history queries

5.5 SEO, Accessibility & Legal

146. SEO: Next.js generateMetadata for all pages; dynamic OG images for scan results (og-image edge function)
147. Structured data: JSON-LD for organization, product, and FAQ pages
148. Accessibility: WCAG 2.1 AA compliance — keyboard navigation, ARIA labels, color contrast ratios
149. i18n: next-intl for English + Hindi initially; architecture supports adding more locales
150. Legal pages: Privacy Policy (GDPR + Indian PDPB compliant), Terms of Service, Cookie Policy, API Terms

151. GDPR: right to erasure (DELETE /api/v1/user endpoint deletes all PII and scan data), data export

5.6 Pricing & Monetization

Plan	Features & Limits
Free	50 scans/month, text + URL analysis, 7-day history, no API access
Pro (₹499/mo)	5,000 scans/month, all signals incl. image + document, 1 year history, 1,000 API calls/day, browser extension Pro features
Enterprise	Unlimited scans, SLA, dedicated support, on-premise ML option, bulk document verification, custom webhook

152. Integrate Razorpay for INR payments (primary market) + Stripe for international

153. Implement usage metering: track per-user scan counts, enforce limits, send upgrade prompts at 80% usage

Launch Checklist: Load test at 10x expected traffic (k6 or Locust). Run full OWASP scan. Legal review of Privacy Policy. Set up customer support (Crisp chat widget). Prepare ProductHunt launch assets. Send beta invites to 50 early users.

Edge Cases & Error Handling

Every user-facing failure must produce a clear, actionable error message. Every system failure must be logged, alerted on, and handled gracefully.

Input Edge Cases

Edge Case	Handling Strategy
URL behind paywall/auth	Detect 401/403 response; return "Content inaccessible" verdict with partial domain-only analysis
URL redirects (shortened links)	Follow up to 5 redirects; log redirect chain; flag suspicious redirect chains (e.g., to different domain)
Very long text (>100k tokens)	Chunk into paragraphs; analyze each chunk; aggregate scores with weighted average
Non-English content	Language detection first; route to multilingual model; flag if model confidence < 0.5 for unsupported language
Password-protected PDFs	Return "Encrypted document" error; prompt user to remove password before upload
Corrupted file upload	Validate file integrity (magic bytes check) immediately; return 422 with clear error
Duplicate scan (same URL/hash)	Return cached result if < 1h old; offer "Re-analyze" button for fresh scan
URL that returns non-HTML (PDF, video)	Detect content-type from headers; route to appropriate analyzer; flag unsupported types
Empty text / whitespace only	Validate minimum 50 characters for text analysis; return 400 with user-friendly message
Image too large (>10MB)	Resize to max 2048px before analysis; log original dimensions; note resizing in evidence

ML / Analysis Edge Cases

Edge Case	Handling Strategy
ML service unavailable	Circuit breaker pattern; return partial score from available signals; never block user with 500
ML inference timeout (>30s)	Async job: return scanId immediately; poll for result; retry failed ML calls up to 3 times

Low-confidence score (40-60 range)	Show "Uncertain" verdict; explain which signals conflict; suggest manual review
All signals absent (e.g., text-only URL)	Only score available signals; weight denominator = sum of present signals only
Model version drift	Tag every scan result with model version; allow historical comparisons; alert on accuracy drops > 5%
TinEye / Google API quota exceeded	Gracefully degrade: skip provenance signal; adjust ensemble weights; log quota hit
EXIF data intentionally stripped	Flag "Missing EXIF" as a weak negative signal (AI images often have no EXIF); note in evidence
AI-generated image with intact EXIF	Some tools add fake EXIF; do not rely solely on EXIF; weight GAN fingerprint higher

Document Verification Edge Cases

Edge Case	Handling Strategy
Document already has a signature	Check if signature is TrueLens-issued; if yes, verify against DB; if external, flag as unknown
Aadhaar QR scan fails	Fall back to visual field analysis; note "QR unreadable" in findings
Invoice with handwritten amounts	OCR with EasyOCR; lower confidence for handwritten fields; flag for manual review
Scanned document (not native PDF)	Detect scan via DPI/image analysis; run OCR pipeline; note lower accuracy in report
Document submitted after manipulation	Hash will not match re-computed hash; flag as tampered; revoke signature if previously signed
Multi-page documents	Analyze each page; flag individual suspicious pages; overall score = weighted average

Security Edge Cases

Edge Case	Handling Strategy
SSRF via URL input	Validate URL: must be http/https, resolve DNS, block RFC 1918 + localhost + link-local ranges
XSS in scan results	All ML output is treated as untrusted text; DOMPurify sanitize before rendering; strict CSP
Malicious file upload (polyglot)	Magic bytes validation + ClamAV scan + MIME re-detection after upload

API key brute force	Rate limit /api/v1 by IP (100 req/min); lock API key after 10 invalid auth attempts in 1h
Replay attacks on webhooks	Include timestamp in webhook payload; reject webhooks older than 5 minutes
Enumeration of scan IDs	Use UUIDs for all IDs; require auth to view scan results unless explicitly made public
DDoS on public endpoints	Cloudflare WAF + rate limiting; /v1/scan requires API key (not accessible anonymously)

Project Timeline

Estimated timeline for a 5-person team working part-time (~20h/week per person). Adjust based on actual availability.

Phase	Duration
Iteration 1 — Prototype	3 weeks
Iteration 2 — Full ML	4 weeks
Iteration 3 — Document Tamper	3 weeks
Iteration 4 — API & Extension	3 weeks
Iteration 5 — Production Polish	3 weeks
Total	~16 weeks (4 months)

Run Iterations 3 and 4 in parallel across sub-teams if bandwidth allows (e.g., 2 people on document workflow, 2 on API + extension, 1 on ML improvements). This compresses timeline to ~12 weeks.

Team Role Assignments (Suggested)

Member	Primary Responsibility
Aryam Agarwal	Frontend Lead — Next.js, UI components, design system, browser extension
Priyanshu Agarwal	ML Lead — model training, FastAPI service, ensemble scoring
Hitarth Singh Rajput	Backend Lead — Express API, BullMQ, database, auth
Akshat Singh	Full Stack — document pipeline, cryptographic signing, provenance
Piyush Kumar	DevOps + API — infrastructure, CI/CD, public API, developer portal

Open Source Resources & References

All resources below are free/open-source or have generous free tiers — buildable with zero initial budget.

ML Models & Datasets (Free)

- roberta-base-openai-detector — HuggingFace, pre-trained GPT-2 detector, fine-tune for modern LLMs
- HC3 Dataset — Human ChatGPT Comparison Corpus for AI text detection training
- raj-tomar001/fake-reviews-dataset — HuggingFace, Amazon/Yelp fake review training data
- CNNDetection — GAN image detection model weights (Gragnaniello et al., GitHub: grip-unina/CNNDetection)
- EasyOCR — Open source OCR, 80+ languages, better than Tesseract for handwriting
- sentence-transformers — multilingual embeddings for semantic similarity

Backend Libraries (Free/OSS)

- BullMQ — Redis-based job queue (MIT license)
- Prisma — TypeScript ORM (Apache 2.0)
- Sharp — Fast Node.js image processing (Apache 2.0)
- exiftool — Perl EXIF tool, wrap via exiftool-vendored npm package
- pdfplumber — Python PDF text/table extraction (MIT)
- pypdf — Python PDF manipulation and watermarking (BSD)

Free API Tiers

- TinEye API — 100 reverse image searches/month free
- Google Fact Check Tools API — free, no quota listed
- Wayback Machine API — fully free, CDX API
- WHOIS API — whoisxmlapi.com has 500 free lookups/month
- PhishTank API — free, updated hourly blocklist
- URLhaus (abuse.ch) — free malicious URL database
- Resend — 3,000 emails/month free tier

Infrastructure (Free/Low-Cost Tiers)

- Vercel — free tier for Next.js frontend, unlimited personal projects
- Railway — \$5/month hobby plan covers API + Redis + PostgreSQL
- Cloudflare — free CDN, SSL, DDoS protection, Workers for edge functions
- Supabase — free PostgreSQL with 500MB storage (alternative to self-hosted Postgres)
- Cloudflare R2 — free S3-compatible storage up to 10GB, no egress fees

Appendix: File Structure

Recommended monorepo layout for the TrueLens project.

```

true lens/
├── apps/
│   ├── web/
│   │   ├── app/
│   │   │   ├── (auth)/
│   │   │   ├── dashboard/
│   │   │   ├── scan/
│   │   │   ├── results/[id]/
│   │   │   ├── documents/
│   │   │   ├── verify/
│   │   │   └── developers/
│   │   ├── components/
│   │   ├── lib/
│   │   └── public/
│   └── api/
│       ├── src/
│       │   ├── routes/
│       │   ├── middleware/
│       │   ├── services/
│       │   ├── workers/
│       │   └── lib/
│       └── prisma/
├── services/
│   └── ml/
│       ├── app/
│       │   ├── routers/
│       │   ├── models/
│       │   └── utils/
│       └── tests/
├── packages/
│   ├── ui/
│   ├── types/
│   └── config/
├── extension/
├── docker-compose.yml
├── .github/workflows/
└── docs/

```